

B Expanded Discussion

The applications in Section 3 show how AutoGen not only enables new applications but also helps renovate existing ones. For example, in A1 (scenario 3), A5, and A6, AutoGen enabled the creation of multi-agent conversations that follow a dynamic pattern instead of a fixed back-and-forth. And in both A5 and A6, humans can participate in the activities together with multiple other AI agents in a conversational manner. Similarly, A1-A4 show how popular applications can be renovated quickly with AutoGen. Despite the complexity of these applications (most of them involve more than two agents or dynamic multi-turn agent cooperation), our AutoGen-based implementation remains simple, demonstrating promising opportunities to build creative applications and a large space for innovation. In reflecting on *why* these benefits can be achieved in these applications with AutoGen, we believe there are a few reasons:

- **Ease of use:** The built-in agents can be used out-of-the-box, delivering strong performance even without any customization. (A1, A3)
- **Modularity:** The division of tasks into separate agents promotes modularity in the system. Each agent can be developed, tested, and maintained independently, simplifying the overall development process and facilitating code management. (A3, A4, A5, and A6)
- **Programmability:** AutoGen allows users to extend/customize existing agents to develop systems satisfying their specific needs with ease. (A1-A6). For example, with AutoGen, the core workflow code in A4 is reduced from over 430 lines to 100 lines, for a 4x saving.
- **Allowing human involvement:** AutoGen provides a native mechanism to achieve human participation and/or human oversight. With AutoGen, humans can seamlessly and optionally cooperate with AIs to solve problems or generally participate in the activity. AutoGen also facilitates interactive user instructions to ensure the process stays on the desired path. (A1, A2, A5, and A6)
- **Collaborative/adversarial agent interactions:** Like many collaborative agent systems (Dong et al., 2023), agents in AutoGen can share information and knowledge, to complement each other’s abilities and collectively arrive at better solutions. (A1, A2, A3, and A4). Analogously, in certain scenarios, some agents are required to work in an adversarial way. Relevant information is shared among different conversations in a controlled manner, preventing distraction or hallucination. (A4, A6). AutoGen supports both patterns, enabling effective utilization and augmentation of LLMs.

B.1 General Guidelines for Using AutoGen

Below we give some recommendations for using agents in AutoGen to accomplish a task.

1. **Consider using built-in agents first.** For example, AssistantAgent is pre-configured to be backed by GPT-4, with a carefully designed system message for generic problem-solving via code. The UserProxyAgent is configured to solicit human inputs and perform tool execution. Many problems can be solved by simply combining these two agents. When customizing agents for an application, consider the following options: (1) human input mode, termination condition, code execution configuration, and LLM configuration can be specified when constructing an agent; (2) AutoGen supports adding instructions in an initial user message, which is an effective way to boost performance without needing to modify the system message; (3) UserProxyAgent can be extended to handle different execution environments and exceptions, etc.; (4) when system message modification is needed, consider leveraging the LLM’s capability to program its conversation flow with natural language.
2. **Start with a simple conversation topology.** Consider using the two-agent chat or the group chat setup first, as they can often be extended with the least code. Note that the two-agent chat can be easily extended to involve more than two agents by using LLM-consumable functions in a dynamic way.
3. Try to **reuse built-in reply methods** based on LLM, tool, or human before implementing a custom reply method because they can often be reused to achieve the goal in a simple way (e.g., the built-in agent GroupChatManager’s reply method reuses the built-in LLM-based reply function when selecting the next speaker, ref. A5 in Section 3).
4. When developing a new application with UserProxyAgent, **start with humans always in the loop**, i.e., `human_input_mode=‘ALWAYS’`, even if the target operation mode is more autonomous. This helps evaluate the effectiveness of AssistantAgent, tuning the prompt, discovering corner cases, and debugging. Once confident with small-scale success, consider setting

human_input_mode = 'NEVER'. This enables LLM as a backend, and one can either use the LLM or manually generate diverse system messages to simulate different use cases.

5. Despite the numerous advantages of AutoGen agents, there could be cases/scenarios where **other libraries/packages could help**. For example: (1) For (sub)tasks that do not have requirements for back-and-forth trouble-shooting, multi-agent interaction, etc., a unidirectional (no back-and-forth message exchange) pipeline can also be orchestrated with LangChain (LangChain, 2023), LlamaIndex (Liu, 2022), Guidance (Guidance, 2023), Semantic Kernel (Semantic-Kernel, 2023), Gorilla (Patil et al., 2023) or low-level inference API ('autogen.oai' provides an enhanced LLM inference layer at this level) (Dibia, 2023). (2) When existing tools from LangChain etc. are helpful, one can use them as tool backends for AutoGen agents. For example, one can readily use tools, e.g., Wolfram Alpha, from LangChain in AutoGen agent. (3) For specific applications, one may want to leverage agents implemented in other libraries/packages. To achieve this, one could wrap those agents as conversable agents in AutoGen and then use them to build LLM applications through multi-agent conversation. (4) It can be hard to find an optimal operating point among many tunable choices, such as the LLM inference configuration. Blackbox optimization packages like 'flaml.tune' (Wang et al., 2021) can be used together with AutoGen to automate such tuning.

B.2 Future Work

This work raises many research questions and future directions and .

Designing optimal multi-agent workflows: Creating a multi-agent workflow for a given task can involve many decisions, e.g., how many agents to include, how to assign agent roles and agent capabilities, how the agents should interact with each other, and whether to automate a particular part of the workflow. There may not exist a one-fits-all answer, and the best solution might depend on the specific application. This raises important questions: For what types of tasks and applications are multi-agent workflows most useful? How do multiple agents help in different applications? For a given task, what is the optimal (e.g., cost-effective) multi-agent workflow?

Creating highly capable agents: AutoGen can enable the development of highly capable agents that leverage the strengths of LLMs, tools, and humans. Creating such agents is crucial to ensuring that a multi-agent workflow can effectively troubleshoot and make progress on a task. For example, we observed that CAMEL, another multi-agent LLM system, cannot effectively solve problems in most cases primarily because it lacks the capability to execute tools or code. This failure shows that LLMs and multi-agent conversations with simple role playing are insufficient, and highly capable agents with diverse skill sets are essential. We believe that more systematic work will be required to develop guidelines for application-specific agents, to create a large OSS knowledge base of agents, and to create agents that can discover and upgrade their skills (Cai et al., 2023).

Enabling scale, safety, and human agency: Section 3 shows how complex multi-agent workflows can enable new applications, and future work will be needed to assess whether scaling further can help solve extremely complex tasks. However, as these workflows scale and grow more complex, it may become difficult to log and adjust them. Thus, it will become essential to develop clear mechanisms and tools to track and debug their behavior. Otherwise, these techniques risk resulting in incomprehensible, unintelligible chatter among agents (Lewis et al., 2017).

Our work also shows how complex, fully autonomous workflows with AutoGen can be useful, but fully autonomous agent conversations will need to be used with care. While the autonomous mode AutoGen supports could be desirable in many scenarios, a high level of autonomy can also pose potential risks, especially in high-risk applications (Amodei et al., 2016; Weld & Etzioni, 1994). As a result, building fail-safes against cascading failures and exploitation, mitigating reward hacking, out of control and undesired behaviors, maintaining effective human oversight of applications built with AutoGen agents will become important. While AutoGen provides convenient and seamless involvement of humans through a user proxy agent, developers and stakeholders still need to understand and determine the appropriate level and pattern of human involvement to ensure the safe and ethical use of the technology (Horvitz, 1999; Amershi et al., 2019).

C Default System Message for Assistant Agent



Figure 5: Default system message for the built-in assistant agent in AutoGen (v0.1.1). This is an example of conversation programming via natural language. It contains instructions of different types, including role play, control flow, output confine, facilitate automation, and grounding.

Figure 5 shows the default system message for the built-in assistant agent in AutoGen (v0.1.1), where we introduce several new prompting techniques and highlight them accordingly. When combining these new prompting techniques together, we can program a fairly complex conversation even with the simplest two-agent conversation topology. This approach tries to exploit the capability of LLMs in implicit state inference to a large degree. LLMs do not follow all the instructions perfectly, so the design of the system needs to have other mechanisms to handle the exceptions and faults. Some instructions can have ambiguities, and the designer should either reduce them for preciseness or intentionally keep them for flexibility and address the different situations in other agents. In general, we observe that GPT-4 follows the instructions better than GPT-3.5-turbo.

D Application Details

A1: Math Problem Solving

Scenario 1: Autonomous Problem Solving. We perform both qualitative and quantitative evaluations in this scenario. For all evaluations, we use GPT-4 as the base model, and pre-install the “sympy” package in the execution environment. We compare AutoGen with the following LLM-based agent systems:

- AutoGPT: The out-of-box AutoGPT is used. We initialize AutoGPT by setting the purpose to “solve math problems”, resulting in a “MathSolverGPT” with auto-generated goals.
- ChatGPT+Plugin: We enable the Wolfram Alpha plugin (a math computation engine) in the OpenAI web client.
- ChatGPT+Code Interpreter: This is a recent feature in OpenAI web client. Note that the above two premium features from ChatGPT require a paid subscription to be accessed and are the most competitive commercial systems.
- LangChain ReAct+Python: We use Python agent from LangChain. To handle parsing errors, we set “handle_parsing_errors=True”, and use the default zero-shot ReAct prompt.
- Multi-Agent Debate (Liang et al., 2023): We modified the code of the multi-agent debate to perform evaluation. By default, there are three agents: an affirmative agent, a negative agent, and a moderator.

We also conducted preliminary evaluations on several other multi-agent systems, including BabyAGI, CAMEL, and MetaGPT. The results indicate that they are not suitable choices for solving math problems out of the box. For instance, when MetaGPT is tasked with solving a math problem, it begins developing software to address the problem, but most of the time, it does not actually solve the problem. We have included the test examples in Appendix E.

Table 2: Qualitative evaluation of two math problems from the MATH dataset within the autonomous problem-solving scenario. Each LLM-based system is tested three times on each of the problems. This table reports the problem-solving correctness and summarizes the reasons for failure.

	Correctness	Failure Reason
AutoGen	3/3	N/A.
AutoGPT	0/3	The LLM gives code without the print function so the result is not printed.
ChatGPT+Plugin	1/3	The return from Wolfram Alpha contains 2 simplified results, including the correct answer, but GPT-4 always chooses the wrong answer.
ChatGPT+Code Interpreter	2/3	Returns a wrong decimal result.
LangChain ReAct	0/3	LangChain gives 3 different wrong answers.
Multi-Agent Debate	0/3	It gives 3 different wrong answers due to calculation errors.

(a) Evaluation on the first problem that asks to simplify a square root fraction.

	Correctness	Failure Reason
AutoGen	2/3	The final answer from code execution is wrong.
AutoGPT	0/3	The LLM gives code without the print function so the result is not printed.
ChatGPT+Plugin	1/3	For one trial, GPT-4 got stuck because it keeps giving wrong queries and has to be stopped. Another trial simply gives a wrong answer.
ChatGPT+Code Interpreter	0/3	It gives 3 different wrong answers.
LangChain ReAct	0/3	LangChain gives 3 different wrong answers.
Multi-Agent Debate	0/3	It gives 3 different wrong answers.

(b) Evaluation on the second number theory problem.

For the qualitative evaluation, we utilize two level-5 problems from the MATH dataset, testing each problem three times. The first problem involves simplifying a square root fraction, and the second

problem involves solving a number theory issue. The correctness counts and reasons for failure are detailed in Table 2. For the quantitative evaluation, we conduct two sets of experiments on the MATH dataset to assess the correctness of these systems: (1) an experiment involving 120 level-5 (the most challenging level) problems, including 20 problems from six categories, excluding geometry, and (2) an experiment on the entire test set, which includes 5000 problems. We exclude AutoGPT from this evaluation as it cannot access results from code executions and does not solve any problems in the qualitative evaluation. Our analysis of the entire dataset reveals that AutoGen achieves an overall accuracy of 69.48%, while GPT-4’s accuracy stands at 55.18%. From these evaluations, we have the following observations regarding the problem-solving success rate and user experience of these systems:

- **Problem-solving success rate:** Results from the quantitative evaluations show that AutoGen can help achieve the highest problem-solving success rate among all the compared methods. The qualitative evaluations elucidate common failure reasons across several alternative approaches. ChatGPT+Code Interpreter fails to solve the second problem, and ChatGPT+Plugin struggles to solve both problems. AutoGPT fails on both problems due to code execution issues. The LangChain agent also fails on both problems, producing code that results in incorrect answers in all trials.
- **Based on the qualitative evaluation, we analyze the user experience concerning the verbosity of the response and the ability of the LLM-based system to run without unexpected behaviors.** ChatGPT+Plugin is the least verbose, mainly because Wolfram queries are much shorter than Python code. AutoGen, ChatGPT+Code Interpreter, and LangChain exhibit similar verbosity, although LangChain is slightly more verbose due to more code execution errors. AutoGPT is the most verbose system owing to predefined steps like THOUGHTS, REASONING, and PLAN, which it includes in replies every time. Overall, AutoGen and ChatGPT+Code Interpreter operate smoothly without exceptions. We note the occurrences of undesired behaviors from other LLM-based systems that could affect user experience: AutoGPT consistently outputs code without the ‘print’ statement and cannot correct this, requiring the user to run them manually; ChatGPT with Wolfram Alpha plugin has the potential to become stuck in a loop that must be manually stopped; and Langchain ReAct could exit with a parse error, necessitating the passing of a ‘handle_parse_error’ parameter.

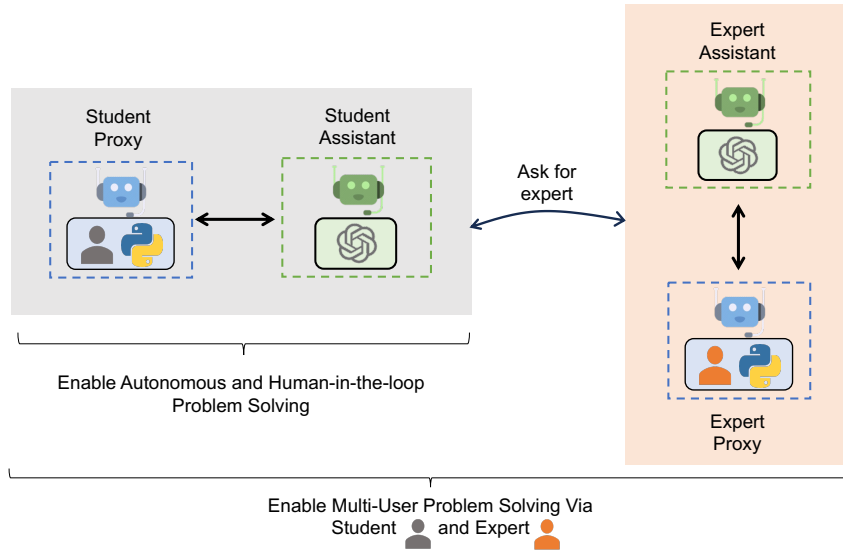


Figure 6: Examples of three settings utilized to solve math problems using AutoGen: (Gray) Enables a workflow where a student collaborates with an assistant agent to solve problems, either autonomously or in a human-in-the-loop mode. (Gray + Orange) Facilitates a more sophisticated workflow wherein the assistant, on the fly, can engage another user termed “expert”, who is in the loop with their own assistant agent, to aid in problem-solving if its own solutions are not satisfactory.

Scenario 2: Human-in-the-loop Problem Solving. For challenging problems that these LLM systems cannot solve autonomously, human feedback during the problem-solving process can be